

LAPORAN TUGAS BESAR 2

IF3270 PEMBELAJARAN MESIN

Convolutional Neural Network dan Recurrent Neural Network



Disusun oleh:

K02 – Kelompok 14

Albertus Christian Poandy	13523077
Grace Evelyn Simon	13523087
Karol Yangqian Poetrachya	13523093

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2025/2026

DAFTAR ISI

DAFTAR ISI	2
DAFTAR GAMBAR	3
DAFTAR TABEL	4
BAB I	
DESKRIPSI PERSOALAN	5
1.1 Image Classification dengan CNN	5
1.2 Image Captioning dengan RNN dan LSTM	6
BAB II	
PENJELASAN IMPLEMENTASI	9
2.1 Kelas Activation	9
2.2 Kelas Linear	9
2.3 Kelas ReLU	10
2.4 Kelas Sigmoid	10
2.5 Kelas Softmax	11
2.6 Kelas Tanh	11
2.7 Kelas Layer	12
2.8 Kelas Model	13
2.9 Kelas Sequential	13
2.10 Kelas Dense	14
2.11 Kelas Conv2DScratch	15
2.12 Kelas LocallyConnected2DScratch	16
2.13 Kelas MaxPooling2DScratch	18
2.14 Kelas AveragePooling2DScratch	18
2.15 Kelas GlobalAveragePooling2DScratch	19
2.16 Kelas GlobalMaxPooling2DScratch	20
2.17 Kelas FlattenScratch	20
2.18 Kelas SimpleRNNScratch	21
2.19 Kelas LSTMScratch	22
2.20 Fungsi Utilitas CNN (utils.py)	23
2.21 Kelas Embedding	24
BAB III	
PENJELASAN FORWARD PROPAGATION	26
3.1 CNN	26
3.2 RNN	28
3.3 LSTM	29
BAB IV	
HASIL PENGUJIAN	32
4.1 Image Classification	32

4.1.1 Pengaruh Jumlah Layer Konvolusi	32
4.1.2 Perbandingan Shared vs Non-Shared Parameter	33
4.1.3 Pengaruh Banyak Filter Per Layer Konvolusi	33
4.1.4 Pengaruh Ukuran Filter Per Layer Konvolusi	34
4.1.5 Pengaruh Jenis Pooling Layer	35
4.1.6 Perbandingan Keras vs from scratch	36
4.2 Image Captioning	36
4.2.1 Perbandingan Jumlah Layer: RNN	37
4.2.2 Perbandingan Jumlah Layer: LSTM	38
4.2.3 Perbandingan Ukuran Hidden State: RNN	38
4.2.4 Perbandingan Ukuran Hidden State: LSTM	39
4.2.5 Perbandingan RNN vs LSTM	39
4.2.6 Perbandingan Keras vs From Scratch	40
4.2.7 Pengaruh Panjang Maksimum Caption Terhadap BLEU-4	41
BAB V	
KESIMPULAN DAN SARAN	43
5.1 Kesimpulan	43
5.2 Saran	44
BAB VI	
PEMBAGIAN TUGAS	45
BAB VII	
REFERENSI	46
BAB VIII	
LAMPIRAN	47

DAFTAR GAMBAR

Gambar 1.1 Arsitektur LeNet-5 Sebagai Model CNN untuk Klasifikasi Gambar Digital	6
(Sumber: GradientBased Learning Applied to Document	6
Recognition,Y. LeCunn et .al)	6
Gambar 1.2 Arsitektur LeNet-5 Sebagai Model CNN untuk Klasifikasi Gambar Digitit	7
Gambar 1.3 Arsitektur LeNet-5 Sebagai Model CNN untuk Klasifikasi Gambar Digit	8
Gambar 3.1 Proses Forward Konvolusi pada CNN	27
(Sumber: https://colah.github.io/posts/2015-08-Understanding-LSTMs/)	27
Gambar 3.2 Satu Sel RNN Sederhana Memiliki Satu Layer	29
(Sumber: https://colah.github.io/posts/2015-08-Understanding-LSTMs/)	29
Gambar 3.3 Satu Sel LSTM Memiliki 4 Layer	30
(Sumber: https://colah.github.io/posts/2015-08-Understanding-LSTMs/)	30
Gambar 4.1.1 Grafik Loss Pengaruh Jumlah Layer Konvolusi	34
Gambar 4.1.2 Grafik Loss Perbandingan Shared vs Non-Shared Parameter	34
Gambar 4.1.3 Grafik Loss Pengaruh Banyak Filter Per Layer Konvolusi	35
Gambar 4.1.4 Grafik Loss Pengaruh Ukuran Filter Per Layer Konvolusi	36
Gambar 4.1.5 Grafik Loss Pengaruh Jenis Pooling Layer	37
Gambar 4.2.1 Grafik Loss Variasi Jumlah Layer dan Ukuran Hidden State untuk RNN	38
Gambar 4.2.2 Grafik Loss Variasi Jumlah Layer dan Ukuran Hidden State untuk LSTM	38
Gambar 4.2.3 Perbandingan Performa Semua Konfigurasi From Scratch	38
Gambar 4.2.4 Analisis Kualitatif Hasil Prediksi RNN dan LSTM	41
Gambar 4.2.5 Perbandingan Hasil Evaluasi Keras dan Model From Scratch	42
Gambar 4.2.6 Grafik Pengaruh Variasi max_length Terhadap Score	43

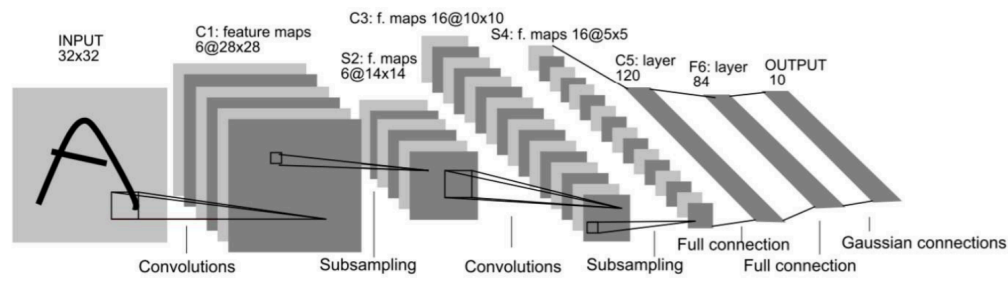
DAFTAR TABEL

Tabel 2.1 Metode Kelas Activation	10
Tabel 2.2 Metode Kelas Linear	10
Tabel 2.3 Metode Kelas ReLU	11
Tabel 2.4 Metode Kelas Sigmoid	11
Tabel 2.5 Metode Kelas Softmax	12
Tabel 2.6 Metode Kelas Tanh	12
Tabel 2.7 Atribut Kelas Layer	13
Tabel 2.8 Metode Kelas Layer	13
Tabel 2.9 Metode Kelas Model	14
Tabel 2.10 Atribut Kelas Sequential	14
Tabel 2.11 Metode Kelas Sequential	14
Tabel 2.12 Atribut Kelas Dense	15
Tabel 2.13 Metode Kelas Dense	15
Tabel 2.14 Atribut Kelas Conv2DScratch	16
Tabel 2.15 Metode Kelas Conv2DScratch	17
Tabel 2.16 Atribut Kelas LocallyConnected2DScratch	18
Tabel 2.17 Metode Kelas LocallyConnected2DScratch	18
Tabel 2.18 Atribut Kelas MaxPooling2dScratch	19
Tabel 2.19 Metode Kelas MaxPooling2DScratch	19
Tabel 2.20 Atribut Kelas AveragePooling2dScratch	20
Tabel 2.21 Metode Kelas AveragePooling2DScratch	20
Tabel 2.22 Metode Kelas GlobalAveragePooling2DScratch	20
Tabel 2.23 Metode Kelas GlobalMaxPooling2DScratch	21
Tabel 2.24 Metode Kelas FlattenScratch	21
Tabel 2.25 Atribut Kelas SimpleRNNScratch	22
Tabel 2.26 Metode Kelas SimpleRNNScratch	22
Tabel 2.27 Atribut Kelas LSTMScratch	23
Tabel 2.28 Metode Kelas LSTMScratch	24
Tabel 2.29 Fungsi Utilitas CNN	25
Tabel 2.30 Atribut Kelas Embedding	26
Tabel 2.31 Metode Kelas Embedding	26

BAB I

DESKRIPSI PERSOALAN

1.1 *Image Classification* dengan CNN



Gambar 1.1 Arsitektur LeNet-5 Sebagai Model CNN untuk Klasifikasi Gambar Digital

(Sumber: GradientBased Learning Applied to Document Recognition, Y. LeCun et al.)

Image classification adalah sebuah task yang melibatkan algoritma *machine learning* untuk memberikan label kelas pada input gambar yang diberikan. Task ini merupakan salah satu masalah dalam *computer vision* dan menjadi landasan bagi banyak aplikasi nyata seperti deteksi objek, diagnosis medis berbasis citra, hingga sistem pengenalan wajah. Pada tugas ini, *image classification* dilakukan menggunakan dataset Intel Image Classification yang terdiri dari sekitar 25.000 gambar pemandangan alam dan perkotaan yang terbagi ke dalam 6 kategori, yakni *buildings*, *forest*, *glacier*, *mountain*, *sea*, dan *street*. Dataset ini telah dibagi ke dalam split *train*, *validation*, dan *test*. Setiap gambar memiliki resolusi 150×150 piksel dengan 3 *channel* warna (RGB).

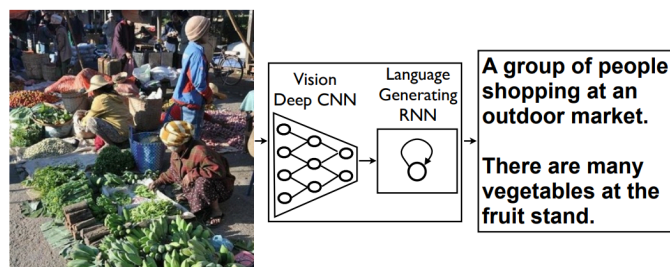
Arsitektur yang digunakan untuk menyelesaikan task ini adalah *Convolutional Neural Network* (CNN). CNN dipilih karena kemampuannya mengekstraksi fitur spasial melalui operasi konvolusi. Lapisan awal menangkap fitur tingkat rendah seperti tepi dan tekstur, sedangkan lapisan yang lebih dalam menangkap fitur semantik yang lebih abstrak. Keunggulan utama CNN dibanding jaringan *fully connected* biasa adalah efisiensi parameter melalui *parameter*

sharing pada filter konvolusi, sehingga model mampu mempelajari pola yang *invariant* terhadap translasi dalam gambar. Arsitektur CNN yang diimplementasikan terdiri dari layer-layer berikut.

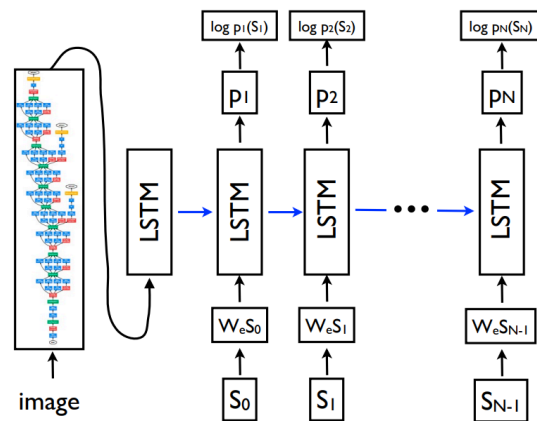
1. Conv2D: layer konvolusi 2D dengan *shared parameters*, dengan satu set *filter* diaplikasikan ke seluruh posisi spasial pada *feature map*.
2. LocallyConnected2D: varian konvolusi tanpa *parameter sharing*, sehingga setiap posisi spasial memiliki bobotnya sendiri.
3. Pooling layer: MaxPooling2D atau AveragePooling2D untuk reduksi dimensi spasial, serta GlobalAveragePooling2D atau GlobalMaxPooling2D untuk reduksi penuh.
4. Flatten: mengubah representasi spasial menjadi vektor 1D.
5. Dense layer: layer *fully connected* untuk menghasilkan prediksi kelas akhir.

Loss function yang digunakan adalah *Sparse Categorical Crossentropy* dengan optimizer Adam, dan metrik evaluasi utama adalah *macro F1-score*. Seluruh operasi *forward propagation* diimplementasikan *from scratch* menggunakan NumPy, dengan bobot yang dimuat dari model Keras yang telah dilatih sebelumnya.

1.2 **Image Captioning dengan RNN dan LSTM**



Gambar 1.2 Arsitektur LeNet-5 Sebagai Model CNN untuk Klasifikasi Gambar Digitit



Gambar 1.3 Arsitektur LeNet-5 Sebagai Model CNN untuk Klasifikasi Gambar Digit

Image captioning adalah *task* yang menggabungkan pemahaman visual dan kemampuan pemrosesan bahasa alami. Diberikan sebuah gambar sebagai *input*, model diharapkan menghasilkan deskripsi teks (*caption*) yang relevan secara otomatis. *Task* ini jauh lebih kompleks dari *image classification* karena outputnya adalah sebuah *sequence* kata dengan panjang yang bervariasi, bukan label tunggal. Dataset yang digunakan adalah Flickr8k, yang terdiri dari 8.092 gambar yang masing-masing dilengkapi dengan 5 *caption* referensi yang ditulis oleh anotator manusia. Dataset ini dibagi menjadi 6.000 gambar untuk *training*, 1.000 untuk *validation*, dan 1.000 untuk *testing*. Arsitektur yang digunakan mengacu pada *paper* Show and Tell (Vinyals et al., 2015), dengan pendekatan *encoder-decoder* sebagai berikut.

1. Encoder: CNN *pretrained* (InceptionV3 atau VGG16 dengan bobot ImageNet yang di-*freeze*) digunakan untuk mengekstraksi *feature vector* dari gambar input.
2. Decoder: Jaringan RNN (SimpleRNN atau LSTM) yang menerima *feature vector* gambar dan menghasilkan *caption* kata per kata secara sekuensial. Metode injeksi yang digunakan adalah *pre-inject*, *feature vector* CNN diproyeksikan ke dimensi *embedding* melalui *dense layer* dan ditempatkan sebagai input pada *timestep* $t = -1$, sebelum token $\langle \text{start} \rangle$. *Hidden state*

awal (h_o) diinisialisasi dengan zeros. Untuk LSTM, c_o juga diinisialisasi dengan zeros. Pada setiap *timestep*, input ke *decoder* adalah *embedding* dari *token ground truth*, bukan prediksi sebelumnya.

Dua varian *decoder* dilatih secara terpisah dan dibandingkan hasilnya, yakni SimpleRNN dan LSTM. SimpleRNN mengolah *sequence* secara berurutan dengan satu *hidden state*, tetapi rentan terhadap masalah *vanishing gradient* untuk *sequence* panjang. LSTM mengatasi kelemahan ini dengan mekanisme *gating* (*input gate*, *forget gate*, *output gate*) dan *cell state* terpisah yang memungkinkan model mempertahankan informasi jangka panjang secara selektif. Evaluasi kualitas *caption* dilakukan menggunakan BLEU-4 score dan METEOR score. Seluruh operasi *forward propagation* dari kedua arsitektur diimplementasikan *from scratch* menggunakan NumPy tanpa bantuan framework *deep learning*, dengan bobot yang dimuat dari model Keras yang telah dilatih sebelumnya.

BAB II

PENJELASAN IMPLEMENTASI

Struktur kelas pada program ini dirancang dengan prinsip *Object-Oriented Programming* (OOP) sehingga lebih modular dan memudahkan pengembangan tiap komponen secara independen. Terdapat kelas abstrak seperti `Activation` dan `Layer` yang mendefinisikan *interface* dasar yang wajib diimplementasikan oleh kelas turunannya.

2.1 Kelas `Activation`

Kelas `Activation` adalah kelas abstrak yang menjadi dasar bagi seluruh fungsi aktivasi. Kelas ini mendefinisikan *interface* berupa metode `forward` dan `derivative` yang wajib diimplementasikan oleh kelas turunannya.

Tabel 2.1 Metode Kelas `Activation`

No.	Metode	Deskripsi
1	<code>forward(self, x)</code>	Menghitung <i>output</i> dari fungsi aktivasi untuk input <i>x</i> .
2	<code>derivative(self, x)</code>	Menghitung turunan fungsi aktivasi terhadap input <i>x</i> .

2.2 Kelas `Linear`

Kelas `Linear` adalah kelas turunan dari `Activation` yang mengimplementasikan fungsi aktivasi linear. Fungsi ini mengembalikan input apa adanya, sehingga sering digunakan pada layer *output* atau sebagai fungsi aktivasi *default*.

Tabel 2.2 Metode Kelas `Linear`

No.	Metode	Deskripsi
1	<code>forward(self, x)</code>	Mengembalikan input <i>x</i> tanpa perubahan.

2	derivative(self, x)	Mengembalikan array satu dengan bentuk sama seperti x.
---	---------------------	--

2.3 Kelas ReLU

Kelas `ReLU` adalah kelas turunan dari `Activation` yang mengimplementasikan fungsi aktivasi *Rectified Linear Unit*. Fungsi ini mengembalikan nilai *input* jika positif dan nol jika negatif. Sering digunakan pada *hidden layer* karena mempercepat konvergensi dan mengurangi masalah *vanishing gradient*.

Tabel 2.3 Metode Kelas `ReLU`

No.	Metode	Deskripsi
1	forward(self, x)	Mengembalikan $\max(0, x)$ untuk setiap elemen x.
2	derivative(self, x)	Mengembalikan 1 jika $x > 0$, jika tidak maka 0.

2.4 Kelas Sigmoid

Kelas `Sigmoid` adalah kelas turunan dari `Activation` yang mengimplementasikan fungsi aktivasi sigmoid, yang memetakan input ke rentang (0, 1). Sering digunakan pada gerbang-gerbang dalam LSTM.

Tabel 2.4 Metode Kelas `Sigmoid`

No.	Metode	Deskripsi
1	forward(self, x)	Menghitung $1/(1 + e^{(-x)})$ untuk setiap elemen x.
2	derivative(self, x)	Menghitung turunan sigmoid, yakni $s(1 - s)$ dengan s adalah hasil <i>forward</i> .

2.5 Kelas Softmax

Kelas `Softmax` adalah kelas turunan dari `Activation` yang mengimplementasikan fungsi aktivasi softmax, yang mengubah vektor input menjadi distribusi probabilitas. Umumnya digunakan pada *output layer* untuk klasifikasi multi-kelas.

Tabel 2.5 Metode Kelas `Softmax`

No.	Metode	Deskripsi
1	<code>forward(self, x)</code>	Menghitung $\exp(x_i) / \sum \exp(x_j)$ untuk setiap elemen pada vektor x dengan stabilisasi numerik.
2	<code>derivative(self, x)</code>	Menghitung matriks Jacobian dari softmax terhadap x.

2.6 Kelas Tanh

Kelas `Tanh` adalah kelas turunan dari `Activation` yang mengimplementasikan fungsi aktivasi hiperbolik tangen, yang memetakan input ke rentang $(-1, 1)$. Digunakan pada gerbang *cell state* LSTM dan pada *hidden layer* SimpleRNN.

Tabel 2.6 Metode Kelas `Tanh`

No.	Metode	Deskripsi
1	<code>forward(self, x)</code>	Menghitung $\tanh(x)$ untuk setiap elemen x.
2	<code>derivative(self, x)</code>	Menghitung $1 - \tanh^2(x)$ untuk setiap elemen x.

2.7 Kelas Layer

Kelas `Layer` adalah kelas abstrak yang menjadi dasar bagi seluruh *layer* dalam implementasi *from scratch*. Kelas ini mendefinisikan *interface* `build`, `forward`, dan `get_output_shape` yang wajib diimplementasikan oleh seluruh kelas turunannya, serta menyediakan *helper* `add_weight` dan `_activation` yang dapat digunakan bersama.

Tabel 2.7 Atribut Kelas `Layer`

No.	Atribut	Deskripsi
1	<code>input_shape</code>	Menyimpan <i>shape input</i> yang diterima <i>layer</i> .
2	<code>output_shape</code>	Menyimpan <i>shape output</i> yang dihasilkan <i>layer</i> .

Tabel 2.8 Metode Kelas `Layer`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Membangun <i>layer</i> berdasarkan <i>shape input</i> .
2	<code>forward(self, x)</code>	Melakukan <i>forward pass</i> .
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output layer</i> .
4	<code>add_weight(self, shape, initializer)</code>	Membuat matriks bobot dengan <i>shape</i> tertentu menggunakan metode inisialisasi yang ditentukan (<i>zeros</i> , <i>ones</i> , atau <i>random</i>).
5	<code>_activation(self, activation_name)</code>	Mengembalikan objek fungsi aktivasi yang sesuai berdasarkan nama <i>string</i> yang diberikan.

2.8 Kelas Model

Kelas `Model` adalah kelas turunan dari `Layer` yang menjadi *base class* bagi seluruh model jaringan saraf. Kelas ini menambahkan metode `predict` sebagai antarmuka prediksi yang memanggil metode `forward`.

Tabel 2.9 Metode Kelas `Model`

No.	Metode	Deskripsi
1	<code>predict(self, x)</code>	Menghasilkan prediksi dari <i>input</i> <code>x</code> dengan memanggil <code>forward</code> .

2.9 Kelas Sequential

Kelas `Sequential` adalah kelas turunan dari `Model` yang merepresentasikan model dengan susunan *layer* berurutan. Setiap *layer* diproses satu per satu dengan *output* satu *layer* menjadi *input layer* berikutnya.

Tabel 2.10 Atribut Kelas `Sequential`

No.	Atribut	Deskripsi
1	<code>layers</code>	<i>List</i> berisi objek-objek <code>Layer</code> yang membentuk urutan model.

Tabel 2.11 Metode Kelas `Sequential`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Membangun seluruh <i>layer</i> secara berurutan, meneruskan <i>shape output</i> satu <i>layer</i> sebagai <i>shape input layer</i> berikutnya.
2	<code>forward(self, x)</code>	Melakukan <i>forward pass</i> secara berurutan melalui seluruh <i>layer</i> dalam daftar.
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> dari <i>layer</i> terakhir dalam daftar.

2.10 Kelas Dense

Kelas `Dense` adalah kelas turunan dari `Layer` yang mengimplementasikan *fully connected layer*. `Layer` ini melakukan operasi $z = Wx + b$ kemudian menerapkan fungsi aktivasi.

Tabel 2.12 Atribut Kelas `Dense`

No.	Atribut	Deskripsi
1	<code>n_neurons</code>	Jumlah <i>neuron</i> pada <i>layer</i> ini.
2	<code>n_inputs</code>	Jumlah fitur <i>input</i> ke <i>layer</i> .
3	<code>weights</code>	Matriks bobot <i>layer</i> dengan <i>shape</i> (<code>n_inputs</code> , <code>n_neurons</code>).
4	<code>biases</code>	Vektor bias <i>layer</i> dengan <i>shape</i> (<code>n_neurons</code> ,).
5	<code>inputs</code>	Menyimpan <i>input</i> yang diterima pada saat <i>forward pass</i> .
6	<code>outputs</code>	Menyimpan <i>output</i> hasil aktivasi pada <i>forward pass</i> .
7	<code>nets</code>	Nilai z (<i>input</i> ke fungsi aktivasi) sebelum diaktivasi.
8	<code>activation_name</code>	Nama <i>string</i> fungsi aktivasi yang digunakan.
9	<code>activation</code>	Objek fungsi aktivasi.

Tabel 2.13 Metode Kelas `Dense`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Membangun <i>layer</i> dengan menginisialisasi fungsi aktivasi dan memvalidasi <i>shape input</i> .
2	<code>forward(self, x)</code>	Melakukan operasi $z = Wx + b$ lalu mengaplikasikan fungsi aktivasi, menyimpan <i>input</i> dan <i>output</i> .
3	<code>get_output_shape</code>	Mengembalikan <i>shape output</i> (<code>batch_size</code> ,

	(self)	n_neurons).
--	--------	-------------

2.11 Kelas Conv2DScratch

Kelas `Conv2DScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan operasi konvolusi 2D dengan *shared parameters* (*parameter sharing*). Satu set bobot kernel diaplikasikan secara bergeser (*sliding*) ke seluruh posisi spasial pada input.

Tabel 2.14 Atribut Kelas `Conv2DScratch`

No.	Atribut	Deskripsi
1	<code>filters</code>	Jumlah <i>filter</i> (<i>channel output</i>) pada <i>layer</i> konvolusi.
2	<code>kernel_size</code>	Ukuran <i>filter</i> konvolusi dalam format (<code>kH</code> , <code>kW</code>).
3	<code>strides</code>	Langkah pergeseran <i>filter</i> dalam format (<code>sh</code> , <code>sw</code>).
4	<code>paddings</code>	Jenis <i>padding</i> yang digunakan (' <code>valid</code> ' atau ' <code>same</code> ').
5	<code>activation_name</code>	Nama <i>string</i> fungsi aktivasi yang digunakan.
6	<code>activation</code>	Objek fungsi aktivasi.
7	<code>kernel</code>	Bobot kernel dengan <i>shape</i> (<code>kH</code> , <code>kW</code> , <code>C_in</code> , <code>C_out</code>).
8	<code>bias</code>	Vektor bias dengan <i>shape</i> (<code>C_out</code> ,).
9	<code>_out_H</code>	Tinggi <i>output</i> setelah konvolusi.
10	<code>_out_W</code>	Lebar <i>output</i> setelah konvolusi.
11	<code>_pad_h</code>	Total <i>padding</i> yang ditambahkan pada dimensi tinggi (untuk <i>padding</i> ' <code>same</code> ').
12	<code>_pad_w</code>	Total <i>padding</i> yang ditambahkan pada dimensi lebar (untuk <i>padding</i> ' <code>same</code> ').

Tabel 2.15 Metode Kelas `Conv2DScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menghitung ukuran <i>output</i> dan jumlah <i>padding</i> berdasarkan <i>shape</i> <code>input</code> , <code>kernel_size</code> , <code>strides</code> , dan <code>padding</code> .
2	<code>forward(self, x)</code>	Melakukan operasi konvolusi 2D. Untuk setiap posisi (<code>i</code> , <code>j</code>), menghitung <i>dot product</i> antara <i>patch input</i> dengan kernel menggunakan <code>np.tensordot</code> , menambahkan bias, lalu mengaplikasikan fungsi aktivasi.
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (<code>N</code> , <code>out_H</code> , <code>out_W</code> , <code>filters</code>).

2.12 Kelas `LocallyConnected2DScratch`

Kelas `LocallyConnected2DScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan konvolusi 2D tanpa *parameter sharing* (*locally connected layer*).

Tabel 2.16 Atribut Kelas `LocallyConnected2DScratch`

No.	Atribut	Deskripsi
1	<code>filters</code>	Jumlah <i>filter</i> (<i>channel output</i>).
2	<code>kernel_size</code>	Ukuran <i>filter</i> dalam format (<code>kH</code> , <code>kW</code>).
3	<code>strides</code>	Langkah pergeseran <i>filter</i> dalam format (<code>sh</code> , <code>sw</code>).
4	<code>activation_name</code>	Nama <i>string</i> fungsi aktivasi yang digunakan.
5	<code>activation</code>	Objek fungsi aktivasi.
6	<code>kernel</code>	Bobot kernel dengan <i>shape</i> (<code>out_H</code> x <code>out_W</code> , <code>kH</code> x <code>kW</code> x <code>C_in</code> , <code>C_out</code>).
7	<code>bias</code>	Vektor bias dengan <i>shape</i> (<code>out_H</code> x <code>out_W</code> , <code>C_out</code>).
8	<code>_out_H</code>	Tinggi <i>output</i> .
9	<code>_out_W</code>	Lebar <i>output</i> .

Tabel 2.17 Metode Kelas `LocallyConnected2DScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menghitung ukuran <i>output</i> berdasarkan <i>shape</i> <code>input</code> , <code>kernel_size</code> , dan <code>strides</code> (hanya mendukung <i>padding</i> 'valid').
2	<code>forward(self, x)</code>	Untuk setiap posisi (<code>i</code> , <code>j</code>), mengambil <i>patch input</i> , meratakannya, lalu mengalikan dengan bobot lokal <code>kernel[idx]</code> dan menambahkan <code>bias[idx]</code> menggunakan <code>np.matmul</code> .
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (<code>N</code> , <code>out_H</code> , <code>out_W</code> , <code>filters</code>).

2.13 Kelas MaxPooling2DScratch

Kelas `MaxPooling2DScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan operasi *max pooling* 2D. Pada setiap *sliding window*, diambil nilai maksimum dari setiap *channel*.

Tabel 2.18 Atribut Kelas `MaxPooling2dScratch`

No.	Atribut	Deskripsi
1	<code>pool_size</code>	Ukuran jendela <i>pooling</i> dalam format (ph, pw).
2	<code>strides</code>	Langkah pergeseran jendela. <i>Default</i> sama dengan <code>pool_size</code> .
3	<code>_out_H</code>	Tinggi <i>output</i> setelah <i>pooling</i> .
4	<code>_out_W</code>	Lebar <i>output</i> setelah <i>pooling</i> .
5	<code>_C</code>	Jumlah <i>channel input/output</i> .

Tabel 2.19 Metode Kelas `MaxPooling2DScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menghitung ukuran <i>output</i> berdasarkan <code>pool_size</code> dan <code>strides</code> .
2	<code>forward(self, x)</code>	Mengambil nilai maksimum dari setiap <i>patch</i> berukuran <code>pool_size</code> menggunakan <code>np.max</code> pada <i>axis</i> spasial.
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (N, <code>out_H</code> , <code>out_W</code> , C).

2.14 Kelas AveragePooling2DScratch

Kelas `AveragePooling2DScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan operasi *average pooling* 2D. Pada setiap *sliding window*, diambil nilai rata-rata dari setiap *channel*.

Tabel 2.20 Atribut Kelas `AveragePooling2dScratch`

No.	Atribut	Deskripsi
1	<code>pool_size</code>	Ukuran jendela <i>pooling</i> dalam format (<code>ph</code> , <code>pw</code>).
2	<code>strides</code>	Langkah pergeseran jendela. <i>Default</i> sama dengan <code>pool_size</code> .
3	<code>_out_H</code>	Tinggi <i>output</i> setelah <i>pooling</i> .
4	<code>_out_W</code>	Lebar <i>output</i> setelah <i>pooling</i> .
5	<code>_C</code>	Jumlah <i>channel input/output</i> .

Tabel 2.21 Metode Kelas `AveragePooling2DScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menghitung ukuran <i>output</i> berdasarkan <code>pool_size</code> dan <code>strides</code> .
2	<code>forward(self, x)</code>	Mengambil nilai rata-rata dari setiap <i>patch</i> berukuran <code>pool_size</code> menggunakan <code>np.mean</code> pada <i>axis</i> spasial.
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (<code>N</code> , <code>out_H</code> , <code>out_W</code> , <code>C</code>).

2.15 Kelas `GlobalAveragePooling2DScratch`

Kelas `GlobalAveragePooling2DScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan reduksi spasial global dengan mengambil rata-rata seluruh nilai spasial per *channel*. Menghasilkan vektor 1D untuk setiap sampel.

Tabel 2.22 Metode Kelas `GlobalAveragePooling2DScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menentukan <i>shape output</i> (<code>N</code> , <code>C</code>) berdasarkan jumlah <i>channel input</i> .

2	<code>forward(self, x)</code>	Menghitung rata-rata seluruh dimensi spasial menggunakan <code>np.mean(x, axis=(1, 2))</code> .
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (N, C).

2.16 Kelas `GlobalMaxPooling2DScratch`

Kelas `GlobalMaxPooling2DScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan reduksi spasial global dengan mengambil maksimum seluruh nilai spasial per *channel*. Menghasilkan vektor 1D untuk setiap sampel.

Tabel 2.23 Metode Kelas `GlobalMaxPooling2DScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menentukan <i>shape output</i> (N, C) berdasarkan jumlah <i>channel input</i> .
2	<code>forward(self, x)</code>	Menghitung nilai maksimum seluruh dimensi spasial menggunakan <code>np.max(x, axis=(1, 2))</code> .
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (N, C).

2.17 Kelas `FlattenScratch`

Kelas `FlattenScratch` adalah kelas turunan dari `Layer` yang me-*reshape* tensor (H, W, C) menjadi vektor 1D dengan urutan *row-major* (C order).

Tabel 2.24 Metode Kelas `FlattenScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Menghitung panjang vektor <i>output</i> sebagai perkalian semua dimensi kecuali <i>batch</i> .
2	<code>forward(self, x)</code>	Me- <i>reshape input</i> menjadi (N, -1) menggunakan urutan C (<i>row-major</i>).

3	get_output_shape (self)	Mengembalikan shape output (N, H x W x C).
---	----------------------------	--

2.18 Kelas SimpleRNNScratch

Kelas `SimpleRNNScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan *Simple Recurrent Neural Network* (`SimpleRNN`) *from scratch*. Pada setiap *timestep*, *hidden state* diperbarui berdasarkan *input* saat ini dan *hidden state* sebelumnya.

Tabel 2.25 Atribut Kelas `SimpleRNNScratch`

No.	Atribut	Deskripsi
1	n_neurons	Ukuran <i>hidden state</i> (jumlah unit RNN).
2	return_sequences	Jika <i>True</i> , mengembalikan <i>output</i> pada setiap <i>timestep</i> . Jika <i>False</i> , hanya <i>timestep</i> terakhir.
3	input_weights	Bobot untuk <i>input</i> (kernel), dimuat dari Keras dengan <i>shape</i> (input_dim, n_neurons).
4	input_bias	Bias RNN, dimuat dari Keras dengan <i>shape</i> (n_neurons,).
5	recurrent_weights	Bobot rekuren (<i>recurrent_kernel</i>), dimuat dari Keras dengan <i>shape</i> (n_neurons, n_neurons).
6	activation_name	Nama string fungsi aktivasi (<i>default</i> : 'relu').
7	activation	Objek fungsi aktivasi.
8	inputs	Menyimpan <i>input</i> pada <i>forward pass</i> .
9	outputs	Menyimpan <i>output</i> pada <i>forward pass</i> .

Tabel 2.26 Metode Kelas `SimpleRNNScratch`

No.	Metode	Deskripsi
1	build(self, input_shape)	Memvalidasi <i>shape input</i> 3D, menginisialisasi fungsi aktivasi, dan menentukan <i>shape</i>

		<i>output</i> berdasarkan <code>return_sequences</code> .
2	<code>forward(self, x)</code>	Melakukan iterasi per <i>timestep</i> , $h_t = \text{activation}(x_t \cdot W_x + h_{\{t-1\}} \cdot W_h + b).$ Mengembalikan seluruh <i>sequence</i> jika <code>return_sequences=True</code> , atau hanya <i>state</i> akhir jika <i>False</i> .
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (N, <i>timesteps</i> , <i>n_neurons</i>) atau (N, <i>n_neurons</i>).

2.19 Kelas `LSTMScratch`

Kelas `LSTMScratch` adalah kelas turunan dari `Layer` yang mengimplementasikan *Long Short-Term Memory (LSTM) from scratch*. LSTM memiliki mekanisme *gating* (*input gate*, *forget gate*, *output gate*) dan *cell state* untuk mengatasi masalah *vanishing gradient* pada *sequence* panjang.

Tabel 2.27 Atribut Kelas `LSTMScratch`

No.	Atribut	Deskripsi
1	<code>n_neurons</code>	Ukuran <i>hidden state</i> dan <i>cell state</i> .
2	<code>return_sequences</code>	Jika <i>True</i> , mengembalikan <i>output</i> pada setiap <i>timestep</i> . Jika <i>False</i> , hanya <i>timestep</i> terakhir.
3	<code>kernel_weights</code>	Bobot untuk <i>input</i> (<i>kernel</i>) dengan <i>shape</i> (<i>input_dim</i> , 4 x <i>n_neurons</i>), mencakup bobot untuk keempat gerbang secara berurutan, yakni <i>i</i> , <i>f</i> , <i>c</i> , <i>o</i> .
5	<code>recurrent_weights</code>	Bobot rekuren (<i>recurrent_kernel</i>) dengan <i>shape</i> (<i>n_neurons</i> , 4 x <i>n_neurons</i>).
9	<code>bias</code>	Bias dengan <i>shape</i> (4 x <i>n_neurons</i> ,) untuk keempat gerbang.

Tabel 2.28 Metode Kelas `LSTMScratch`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Memvalidasi <i>shape input</i> 3D, menginisialisasi fungsi aktivasi, dan menentukan <i>shape output</i> berdasarkan <code>return_sequences</code> .
2	<code>forward(self, x)</code>	Melakukan iterasi per <i>timestep</i> , $h_t = activation(x_t \cdot W_x + h_{\{t-1\}} \cdot W_h + b).$ Mengembalikan seluruh sequence jika <code>return_sequences=True</code> , atau hanya <i>state</i> akhir jika <i>False</i> .
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (N, <i>timesteps</i> , <i>n_neurons</i>) atau (N, <i>n_neurons</i>).
4	<code>set_kernel(self, weights)</code>	<i>Setter</i> untuk bobot kernel.
5	<code>set_recurrent_kernel(self, weights)</code>	<i>Setter</i> untuk bobot <i>recurrent</i> kernel.
6	<code>set_bias(self, bias)</code>	<i>Setter</i> untuk bias.
7	<code>get_kernel_i(self)</code>	Mengembalikan bobot kernel untuk <i>input gate</i> .
8	<code>get_kernel_f(self)</code>	Mengembalikan bobot kernel untuk <i>forget gate</i> .
9	<code>get_kernel_o(self)</code>	Mengembalikan bobot kernel untuk <i>output gate</i> .
10	<code>get_kernel_c(self)</code>	Mengembalikan bobot kernel untuk <i>cell gate</i> .

2.20 Fungsi Utilitas CNN (`utils.py`)

Selain kelas-kelas `layer`, terdapat tiga fungsi utilitas untuk pemrosesan data gambar yang diimplementasikan menggunakan PIL/Pillow dan NumPy tanpa Keras *preprocessing*.

Tabel 2.29 Fungsi Utilitas CNN

No.	Fungsi	Deskripsi
1	<code>load_image(path, target_size)</code>	Memuat satu gambar dari <i>file path</i> menggunakan <code>PIL.Image.open</code> , melakukan <i>resize</i> ke <code>target_size</code> , mengonversi ke NumPy <i>array</i> , dan menormalisasi nilai piksel ke rentang <code>[0, 1]</code> . Mengembalikan <i>array</i> dengan shape <code>(H, W, 3)</code> .
2	<code>load_batch(paths, target_size)</code>	Memuat dan memproses sekumpulan gambar dari <i>list file path</i> menggunakan <code>load_image</code> , lalu menyusunnya menjadi satu NumPy <i>array</i> dengan shape <code>(N, H, W, 3)</code> .
3	<code>extract_features(paths, keras_model, output_path, target_size, batch_size)</code>	Menerima <i>list path</i> gambar dan model Keras (CNN <i>encoder, frozen</i>), mengekstraksi <i>feature vector</i> secara <i>batch</i> , lalu menyimpan hasilnya ke <i>disk</i> dalam format <code>.npy</code> . Jika <i>file</i> sudah ada, langsung memuat dari <i>disk</i> tanpa mengekstraksi ulang.

2.21 Kelas `Embedding`

Kelas `Embedding` adalah kelas turunan dari `Layer` yang mengimplementasikan *embedding layer* untuk mengubah indeks token *integer* menjadi vektor representasi padat (*dense vector*). *Layer* ini digunakan dalam *pipeline image captioning* untuk memetakan setiap token kata ke ruang vektor berdimensi `output_dim`.

Tabel 2.30 Atribut Kelas `Embedding`

No.	Atribut	Deskripsi
1	<code>input_dim</code>	Ukuran kosakata, jumlah token unik yang mungkin menjadi <i>input</i> .
2	<code>output_dim</code>	Dimensi vektor <i>embedding</i> yang dihasilkan untuk setiap token.
3	<code>embeddings</code>	Matriks bobot <i>embedding</i> dengan <i>shape</i> (<code>input_dim</code> , <code>output_dim</code>), dimuat dari Keras.

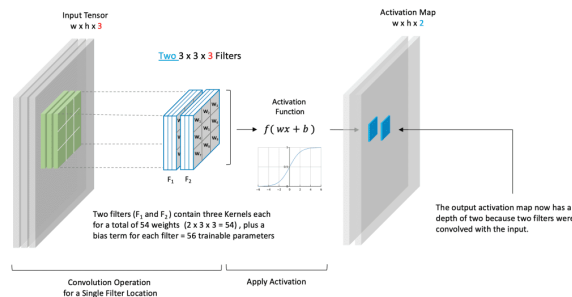
Tabel 2.31 Metode Kelas `Embedding`

No.	Metode	Deskripsi
1	<code>build(self, input_shape)</code>	Memvalidasi <i>shape input</i> 2D (<code>batch_size</code> , <code>sequence_length</code>), menginisialisasi matriks <i>embedding</i> jika belum ada, dan menetapkan <i>shape output</i> (<code>batch_size</code> , <code>sequence_length</code> , <code>output_dim</code>).
2	<code>forward(self, x)</code>	Melakukan <i>lookup</i> pada matriks <i>embedding</i> menggunakan indeks token <i>integer</i> dari input <code>x</code> . Kemudian, memvalidasi bahwa semua indeks berada dalam rentang <code>[0, input_dim)</code> . Mengembalikan <i>array</i> berukuran (<code>N</code> , <code>seq_len</code> , <code>output_dim</code>).
3	<code>get_output_shape(self)</code>	Mengembalikan <i>shape output</i> (<code>batch_size</code> , <code>sequence_length</code> , <code>output_dim</code>).

BAB III

PENJELASAN FORWARD PROPAGATION

3.1 CNN



Gambar 3.1 Proses *Forward* Konvolusi pada CNN

(Sumber: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)

Dalam konteks model berbasis CNN, *forward propagation* melibatkan proses transformasi data *input* berupa piksel gambar melalui serangkaian lapisan (*layers*) untuk menghasilkan prediksi atau fitur pada lapisan *output*. Proses ini melibatkan operasi linear (*convolution*), non linear (fungsi aktivasi atau tahap *detector*), dan *downsampling* (tahap *pooling*).

Input layer menerima data masukan piksel gambar dengan dimensi W (lebar) x H (tinggi) x C (channel). Dimensi C merepresentasikan jumlah *channel* warna pada gambar, misalnya gambar dengan format RGB memiliki nilai C = 3.

Setelah melalui lapisan input, data diteruskan ke *hidden layer*. Sebuah *hidden layer* pada CNN terdiri dari 3 tahap: konvolusi, detektor, dan *pooling*. Pada tahap konvolusi, filter/kernel dengan ukuran yang lebih kecil tapi memiliki jumlah *channel* yang sama dengan input digunakan untuk mentransformasi fitur masukan secara linear menjadi matriks baru. Tahap konvolusi bisa memiliki beberapa filter dengan bobot yang masing-masing bisa dilatih. Sebuah segmen pada fitur input diterapkan operasi perkalian dengan bobot kernel dan dijumlahkan dengan bias lalu kernel digeser seperti mekanisme *sliding window* untuk mengoperasikan

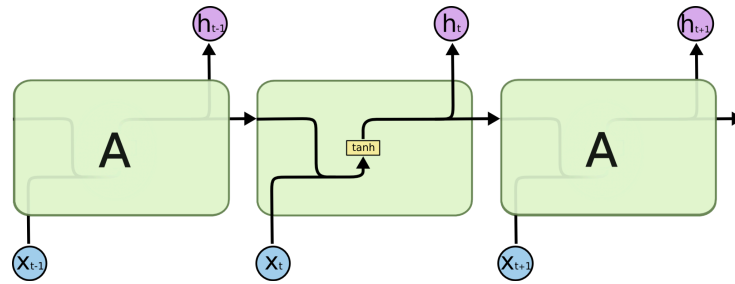
segmen berikutnya dengan cara yang sama. Bobot kernel yang sama digunakan untuk mengoperasikan setiap segmen masukan merupakan karakteristik *shared parameters* pada model berbasis CNN. Karena satu iterasi operasi konvolusi hanya melibatkan sejumlah fitur yang saling berdekatan secara spasial, model berbasis CNN juga disebut memiliki sifat *local connectivity*.

Matriks hasil operasi konvolusi diteruskan ke tahap detektor untuk diterapkan fungsi aktivasi. Salah satu fungsi aktivasi yang umum dan populer digunakan pada CNN adalah ReLU, namun fungsi aktivasi lain bisa digunakan juga sebagai substitusi. Tahap ini menghadirkan nonlinearitas agar model bisa mempelajari pola yang lebih kompleks. Fungsi aktivasi diterapkan secara *element-wise* sehingga menghasilkan output matriks dengan ukuran yang sama dengan tahap konvolusi untuk diteruskan ke tahap *pooling*.

Pada tahap *pooling*, keluaran dari tahap detektor diterapkan *downsampling* agar model dapat melakukan generalisasi dengan lebih baik dan menangani *overfitting*. Terdapat beberapa metode seperti *max pooling*, *average pooling*, dan L2-norm *pooling*. Pada *max pooling* dengan ukuran filter 2×2 , dipilih maksimum dari setiap segmen 2×2 dari fitur hasil detektor sebagai elemen pada matriks output hasil *layer* konvolusi yang disebut *feature map*.

Model berbasis CNN bisa terdiri dari beberapa *hidden layer* konvolusi yang berantai dan masing-masing menghasilkan *feature map* secara bertahap untuk mengekstrak pola-pola pada berbagai abstraksi.

3.2 RNN



Gambar 3.2 Satu Sel RNN Sederhana Memiliki Satu Layer

(Sumber: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)

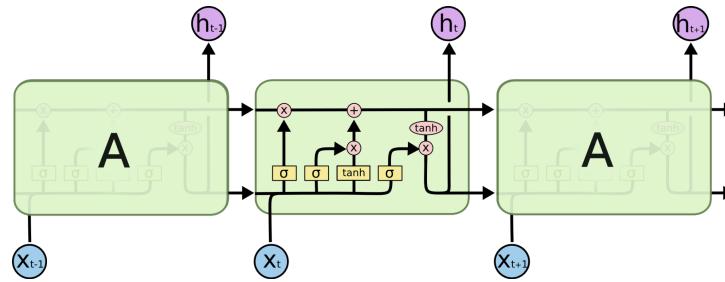
Model berbasis RNN dirancang untuk dapat menangani prediksi data sekuensial dengan cara memasukkan hasil prediksi sebelumnya sebagai input dari proses prediksi berikutnya. Sebuah sel RNN mengeluarkan hasil *hidden state* sebagai hasil operasi linear antara fitur input, bobot, dan bias. *Hidden state* dari hasil operasi pertama digunakan kembali sebagai masukan pada *timestep* berikutnya.

Sebuah sel RNN memiliki 2 jenis bobot, yakni bobot *input-to-hidden* (W_{xh}) dan bobot *hidden-to-hidden* (W_{hh}). Nilai kedua matriks bobot ini tidak berubah sepanjang setiap *timestep*. Untuk setiap *timestep*, diterapkan operasi linear berikut:

$$h^{(t)} = f_h(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h)$$

dengan f_h adalah fungsi aktivasi, $x^{(t)}$ adalah fitur input pada *timestep* t , $h^{(t-1)}$ adalah *hidden state* dari *timestep* sebelumnya, dan b_h adalah bias. Fungsi aktivasi yang digunakan pada *layer* internal RNN adalah hiperbolik tangen ($\tanh$). Pemilihan fungsi aktivasi ini dilakukan karena $\tanh$ dapat memberikan efek kontribusi positif dan negatif pada memori RNN, berbeda dengan sigmoid dan ReLU yang mengembalikan rentang nilai 0 sampai 1.

3.3 LSTM



Gambar 3.3 Satu Sel LSTM Memiliki 4 Layer

(Sumber: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)

Arsitektur LSTM dirancang sebagai pengembangan lanjutan dari RNN dalam menangani data sekuensial. Berbeda dengan RNN, LSTM memiliki 4 *layer* internal dengan bobot yang saling independen namun terlibat dalam beberapa operasi yang sama untuk menghasilkan beberapa variabel inti. Keempat *layer* tersebut adalah *forget gate*, *input gate*, *cell candidate*, dan *output gate*. Berikut adalah penjelasan setiap komponen tersebut.

1. Forget Gate

Gerbang ini berfungsi untuk menentukan informasi mana dari *cell state timestep* sebelumnya yang perlu dibuang/dilupakan karena tidak relevan untuk melakukan prediksi pada *timestep* saat ini. Proses ini dilakukan dengan menerapkan operasi perkalian *element-wise* antara hasil *forget gate* dengan *cell state* dari *timestep* sebelumnya. Dengan operasi ini, informasi dari *cell state* dipertahankan dengan rentang faktor 0 sampai 1. *Forget gate* sendiri dihasilkan dengan persamaan linear berikut:

$$f_t = \sigma(W_{x_f}x^{(t)} + W_{h_f}h^{(t-1)} + b_f)$$

2. Input Gate

Setelah *cell state* diperbarui dengan melupakan beberapa informasi yang sudah tidak relevan, gerbang *input* digunakan untuk menambah informasi baru untuk disimpan ke dalam *cell state*. *Input gate* menggunakan fungsi aktivasi sigmoid yang memiliki rentang keluaran 0 sampai 1 untuk menentukan informasi apa saja dari *cell candidate* yang perlu ditambahkan ke *cell state*. Persamaan yang digunakan untuk *input gate* adalah sebagai berikut:

$$i_t = \sigma(W_{xi}x^{(t)} + W_{hi}h^{(t-1)} + b_i)$$

3. Cell Candidate

Cell candidate adalah informasi yang diekstrak dari fitur input pada *timestep* saat ini yang berpotensi untuk ditambahkan ke dalam memori *cell state*. Kalkulasi *cell candidate* menggunakan fungsi aktivasi tanh agar informasi baru bisa memberi pengaruh arah positif atau negatif terhadap memori jangka panjang. Persamaan *cell candidate* adalah sebagai berikut:

$$\hat{C}_t = \tanh(W_{xc}x^{(t)} + W_{hc}h^{(t-1)} + b_c)$$

4. Output Gate

Output gate berfungsi untuk menghasilkan *hidden state* yang baru menurut *cell state* yang telah dibuat pada *timestep* saat ini. Nilai dari *output gate* dikalkulasi dengan persamaan berikut:

$$o_t = \sigma(W_{xo}x^{(t)} + W_{ho}h^{(t-1)} + b_o).$$

Untuk menghasilkan *hidden state* baru yang akan diteruskan ke *timestep* dan *layer* berikutnya, *cell state* saat ini perlu dihitung dengan persamaan berikut:

$$C_t = f_t \odot C_{t-1} \oplus i_t \odot \hat{C}_t$$

dengan $\odot$ adalah operator perkalian *element-wise* dan $\oplus$ adalah operator penjumlahan *element-wise*. Setelah mendapatkan *cell state*, *hidden state* yang baru dihitung menggunakan persamaan berikut:

$$h_t = o_t \odot \tanh(C_t).$$

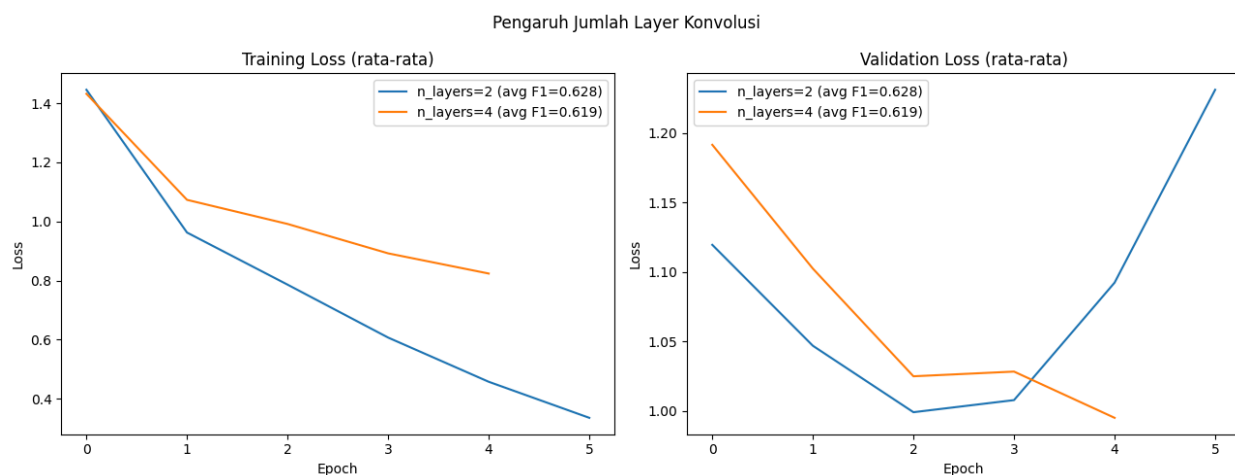
BAB IV

HASIL PENGUJIAN

4.1 *Image Classification*

4.1.1 Pengaruh Jumlah *Layer* Konvolusi

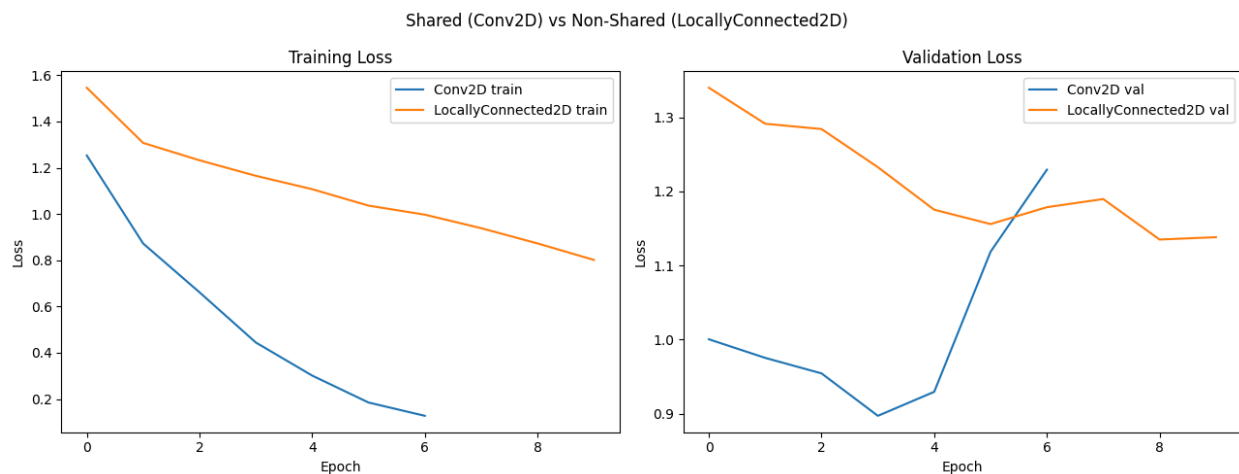
Berdasarkan pengujian dengan variasi jumlah *layer* konvolusi sebanyak 2 dan 4 *layer* (dengan mempertahankan filter [32, 64], kernel 3×3 , dan *max pooling*), diperoleh rata-rata macro F1-score pada data uji berturut-turut sebesar 0.6724 (ID 0) dan 0.6542 (ID 8). Penambahan jumlah *layer* konvolusi dari 2 menjadi 4 justru menurunkan performa model. Meskipun *layer* yang lebih dalam secara teori mampu mengekstraksi hierarki fitur yang lebih abstrak, penambahan *layer* tanpa diimbangi oleh regularisasi yang kuat atau jumlah data yang sangat masif menyebabkan model lebih cepat mengalami overfitting serta meningkatkan waktu komputasi.. Hal ini terlihat dari validation loss pada grafik yang kemungkinan lebih buruk/tidak stabil.



Gambar 4.1.1 Grafik Loss Pengaruh Jumlah *Layer* Konvolusi

4.1.2 Perbandingan Shared vs Non-Shared Parameter

Model dengan shared parameter (Conv2D arsitektur terbaik) memperoleh macro F1-score sebesar 0.6724 dengan jumlah parameter sebanyak 412,614, sedangkan model non-shared parameter (LocallyConnected2D) memperoleh skor 0.5094 dengan jumlah parameter yang jauh lebih besar yakni 5,152,262. Model Conv2D terbukti lebih efisien dan memiliki performa generalisasi yang lebih baik. Adanya parameter sharing membuat model memiliki sifat *translation invariance*, di mana model dapat mengenali objek (misal: bangunan atau hutan) tidak peduli di mana posisi spasial objek tersebut dalam gambar. Sebaliknya, arsitektur non-shared memakan memori yang sangat besar, memperlambat waktu training, dan gagal mengenali pola yang sama jika posisinya bergeser, sehingga rentan terhadap overfitting.

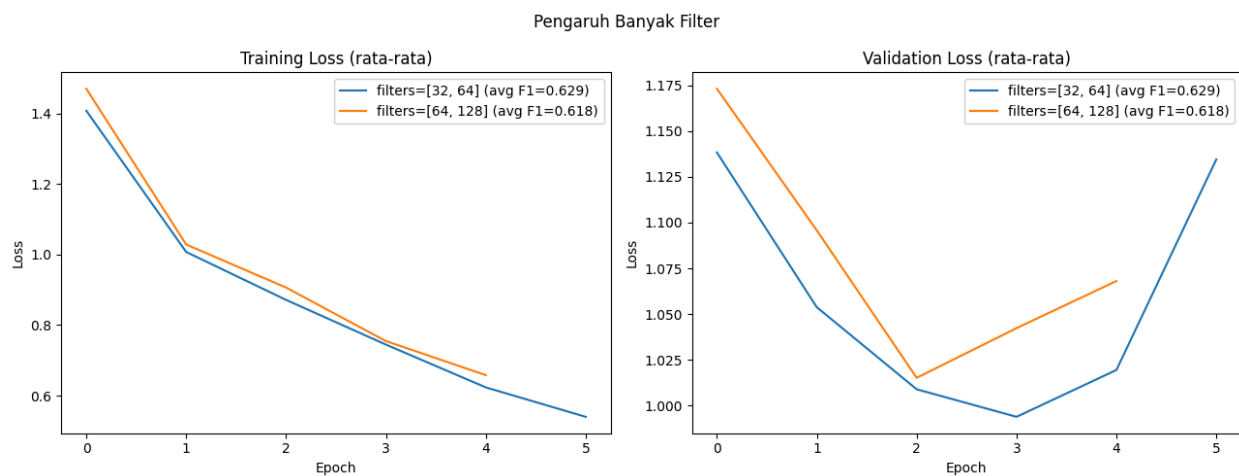


Gambar 4.1.2 Grafik Loss Perbandingan Shared vs Non-Shared Parameter

4.1.3 Pengaruh Banyak Filter Per *Layer* Konvolusi

Pengujian dilakukan dengan variasi kombinasi filter [32, 64] dan [64, 128] (pada model 2 *layer*, kernel 3×3, *max pooling*). Kombinasi filter

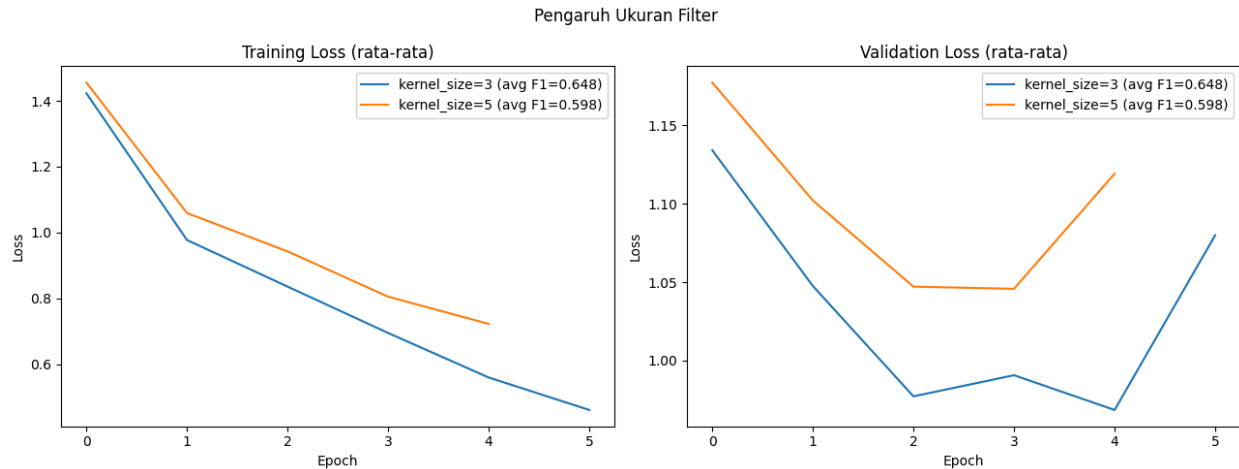
[32, 64] memberikan macro F1-score tertinggi sebesar 0.6724 (ID 0), sedangkan peningkatan filter menjadi [64, 128] menurunkan skor drastis menjadi 0.6111 (ID 4). Meningkatkan jumlah filter memperlebar dimensi representasi fitur, sehingga model memiliki kapasitas belajar yang lebih besar. Namun, jumlah filter yang terlalu berlebihan seperti pada variasi [64, 128] justru membuat model menghafal noise spesifik pada training data alih-alih mempelajari pola umum, sehingga menyebabkan performa turun pada data test akibat overfitting.



Gambar 4.1.3 Grafik Loss Pengaruh Banyak Filter Per Layer Konvolusi

4.1.4 Pengaruh Ukuran Filter Per *Layer* Konvolusi

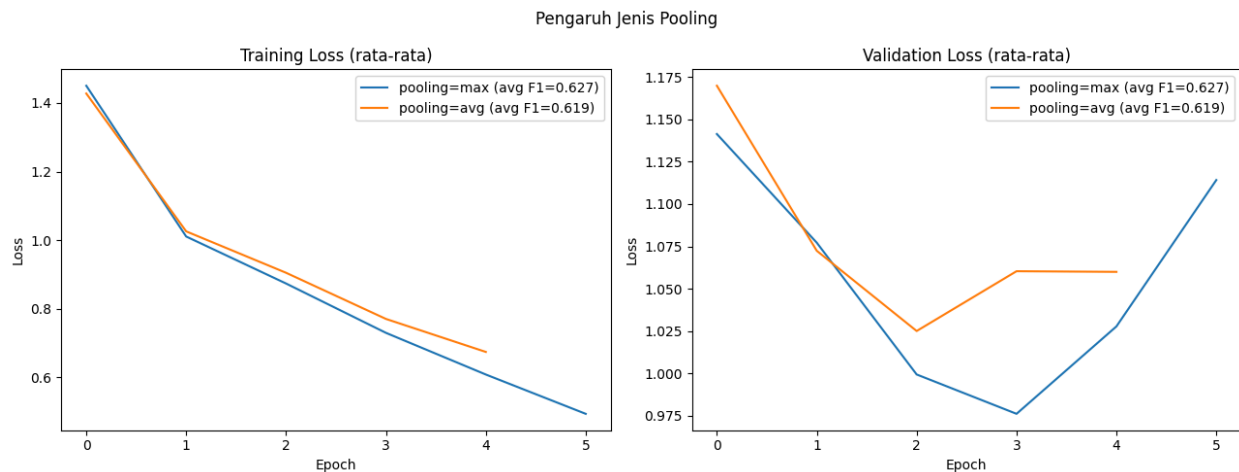
Dari variasi ukuran filter 3×3 dan 5×5 (pada model 2 *layer*, filter [32, 64], *max pooling*), arsitektur dengan ukuran filter 3×3 memberikan hasil terbaik dengan skor 0.6724 (ID 0), dibandingkan filter 5×5 yang hanya mendapat 0.6323 (ID 2). Ukuran filter yang kecil (3×3) terbukti lebih optimal karena mampu menangkap fitur lokal (fine-grained details) dengan sangat tajam dan menggunakan lebih sedikit parameter. Sebaliknya, ukuran filter yang lebih besar (5×5) menangkap konteks spasial yang lebih luas secara langsung tetapi kehilangan detail spasial halus pada gambar, yang berujung pada penurunan F1-score.



Gambar 4.1.4 Grafik Loss Pengaruh Ukuran Filter Per *Layer* Konvolusi

4.1.5 Pengaruh Jenis *Pooling Layer*

Pada variasi baseline yang sama, model dengan *max pooling* menghasilkan macro F1-score 0.6724 (ID 0), sedangkan *average pooling* menghasilkan skor 0.6373 (ID 1). *Max pooling* terbukti memberikan hasil yang lebih baik untuk task klasifikasi gambar ini. Hal ini terjadi karena *max pooling* secara langsung mempertahankan respons fitur yang paling dominan/kuat (misalnya deteksi tepi yang tegas) yang krusial dalam mendiskriminasi kelas. Sebaliknya, *average pooling* meratakan (*smoothing*) representasi piksel di dalam *window*, sehingga batas fitur yang tegas kehilangan ketajamannya.

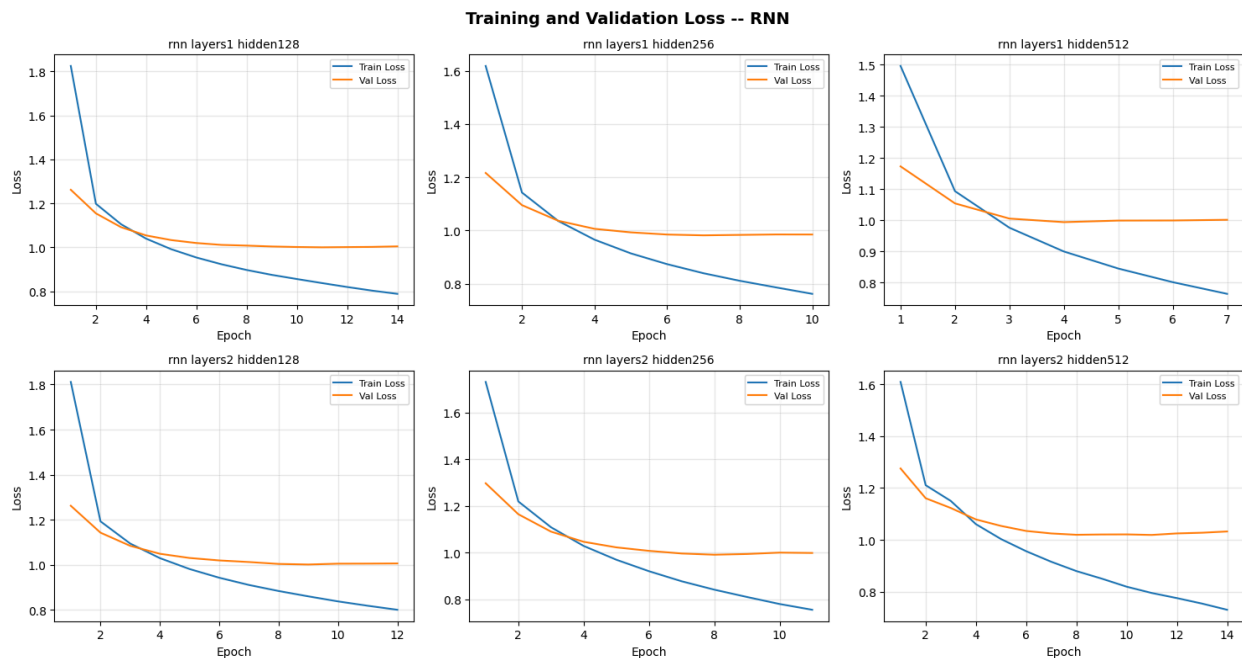


Gambar 4.1.5 Grafik Loss Pengaruh Jenis Pooling Layer

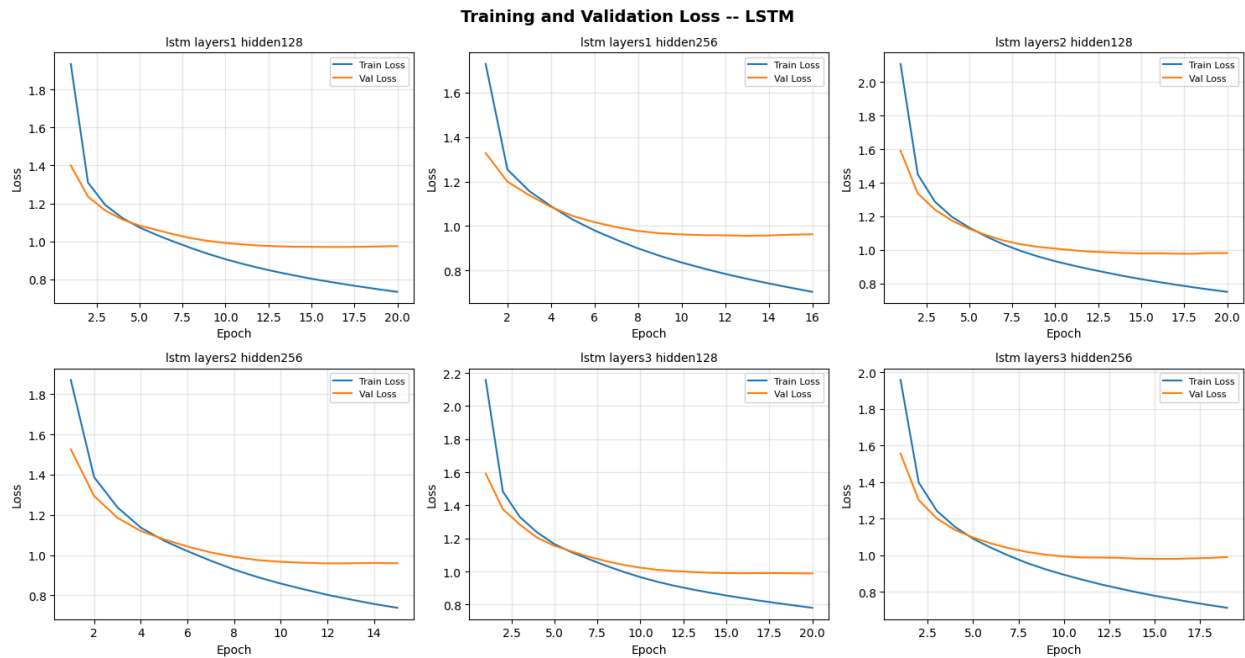
4.1.6 Perbandingan Keras vs from scratch

Hasil eksekusi komputasi metrik pada arsitektur terbaik menunjukkan bahwa model Keras mendapatkan macro F1-score 0.6724, dan implementasi from scratch mendapatkan F1-score 0.6724. Kedua implementasi menghasilkan akurasi/skor yang identik. Hal ini membuktikan bahwa persamaan matematis propagasi maju untuk operasi konvolusi spasial dan pemrosesan lokal telah diimplementasikan dengan benar pada NumPy.

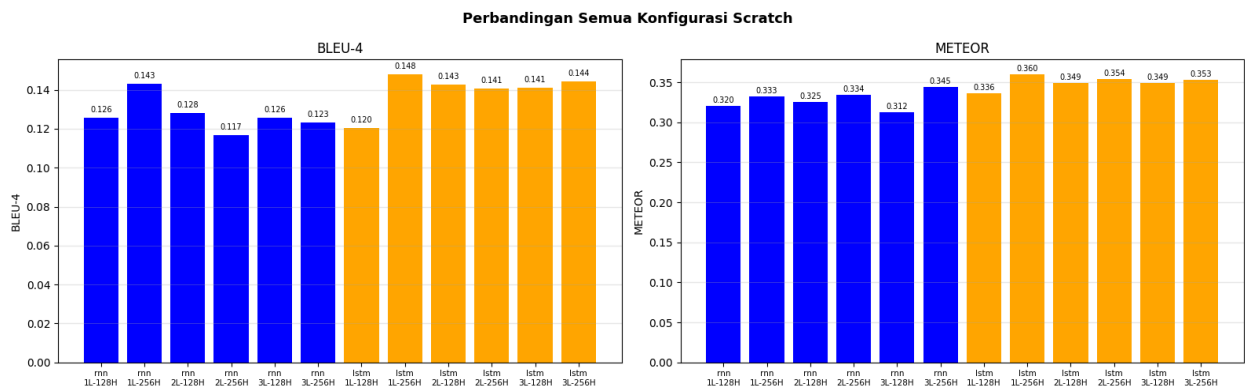
4.2 Image Captioning



Gambar 4.2.1 Grafik Loss Variasi *Jumlah Layer* dan Ukuran *Hidden State* untuk RNN



Gambar 4.2.2 Grafik Loss Variasi *Jumlah Layer* dan Ukuran *Hidden State* untuk LSTM



Gambar 4.2.3 Perbandingan Performa Semua Konfigurasi *From Scratch*

4.2.1 Perbandingan Jumlah *Layer*: RNN

Berdasarkan pengujian pada arsitektur SimpleRNN dengan hidden state 256, variasi sebanyak 1, 2, dan 3 *layer* memberikan BLEU-4 score berturut-turut sebesar 0.1432, 0.1169, dan 0.1231. Penambahan *layer* recurrent pada SimpleRNN tidak meningkatkan performa, melainkan menurunkannya. Arsitektur 1 *layer* terbukti memberikan

hasil terbaik. Hal ini menunjukkan kelemahan dari arsitektur SimpleRNN, yakni masalah *vanishing gradient*. Semakin dalam *layer* jaringannya, informasi fitur awal dari CNN yang sangat penting perlahan memudar karena perkalian matriks berulang, sehingga performanya tertinggal dibandingkan versi dangkalnya (1 *layer*).

4.2.2 Perbandingan Jumlah Layer: LSTM

Pengujian pada arsitektur LSTM dengan ukuran *hidden state* 256 menggunakan 1, 2, dan 3 *layer* memberikan BLEU-4 score berturut-turut sebesar 0.1481, 0.1406, dan 0.1442. Sama seperti pola pada RNN, penambahan *layer* pada LSTM dari 1 menjadi 2 atau 3 *layer* ternyata tidak memberikan peningkatan kualitas *captioning*. Meskipun LSTM memiliki mekanisme *gating* untuk mengatasi *vanishing gradient*, kompleksitas model yang terlalu tinggi (akibat bertambahnya *layer*) membuat model rentan mengalami *overfitting* pada dataset berukuran terbatas seperti Flickr8k. Oleh karena itu, representasi 1 *layer* sudah cukup optimal untuk mempelajari hierarki *sequence* pada kasus ini.

4.2.3 Perbandingan Ukuran Hidden State: RNN

Pengujian pada arsitektur SimpleRNN (1 *layer*) dengan ukuran *hidden state* 128 memperoleh BLEU-4 sebesar 0.1255, sedangkan ukuran 256 memperoleh skor 0.1432. Peningkatan ukuran *hidden state* dari 128 ke 256 memberikan dampak positif pada performa. Ukuran *hidden state* yang lebih besar memperluas kapasitas memori model, sehingga kemampuannya dalam mengingat konteks kalimat menjadi lebih baik.

4.2.4 Perbandingan Ukuran Hidden State: LSTM

Arsitektur LSTM (1 *layer*) dengan ukuran *hidden state* 128 memperoleh BLEU-4 sebesar 0.1205, sedangkan ukuran 256 memperoleh peningkatan skor menjadi 0.1481. Peningkatan ukuran *hidden state* pada LSTM berdampak signifikan pada peningkatan skor BLEU-4 dan METEOR. Kapasitas memori (*cell state* dan *hidden state*) yang lebih besar memungkinkan gerbang-gerbang pada LSTM (khususnya *input gate* dan *forget gate*) untuk mengkodekan representasi semantik dari *feature vector* gambar ke dalam ruang kata yang lebih kaya dan presisi.

4.2.5 Perbandingan RNN vs LSTM

Secara keseluruhan, konfigurasi terbaik LSTM (1 *layer*, hidden 256) mencapai BLEU-4 0.1481 dan METEOR 0.3605, mengungguli konfigurasi terbaik SimpleRNN (1 *layer*, hidden 256) yang mendapat BLEU-4 0.1432 dan METEOR 0.3327. Waktu eksekusi LSTM (114.6 detik) lebih lama dibandingkan RNN (83.2 detik). Arsitektur *pre-inject* menempatkan fitur gambar pada *timestep* terawal. SimpleRNN lebih cepat "melupakan" fitur gambar ini di akhir proses *decoding* akibat keterbatasan memorinya. Di sisi lain, LSTM menggunakan memori internal dengan mekanisme gerbang logis. *Forget gate* memungkinkan LSTM mengontrol retensi nilai fitur gambar dari awal hingga akhir, membuat susunan kata LSTM lebih padu dan terikat langsung dengan *ground truth* visual (seperti yang terlihat pada analisis kualitatif), dibandingkan RNN yang rentan kehilangan konteks. Performa yang lebih baik ini harus dibayar dengan waktu komputasi yang lebih lambat akibat operasi matematis 4 gerbang internal LSTM.

[TINGGI] Score: 1.0000



Qualitative Analysis — RNN ('rnn', 1, 256) vs LSTM ('lstm', 1, 256)

```
RNN :  
a black and white dog is running through a field  
LSTM:  
a black and white dog is running through a field  
GT :  
a black and white dog is running in the grass  
a black and white dog running away across the grass
```

[TINGGI] Score: 0.9457



```
RNN :  
a black dog is running through a field  
LSTM:  
a black dog is running through a field  
GT :  
a black dog is running through a swampy area with tall reeds  
a large dog runs through a field in the rain
```

[SEDANG] Score: 0.1368



```
RNN :  
a dog is running through a field  
LSTM:  
a dog is running through the water  
GT :  
a brown and black dog runs down a sandy beach  
an animal running through the sand at the beach
```

[RENDAH] Score: 0.0138



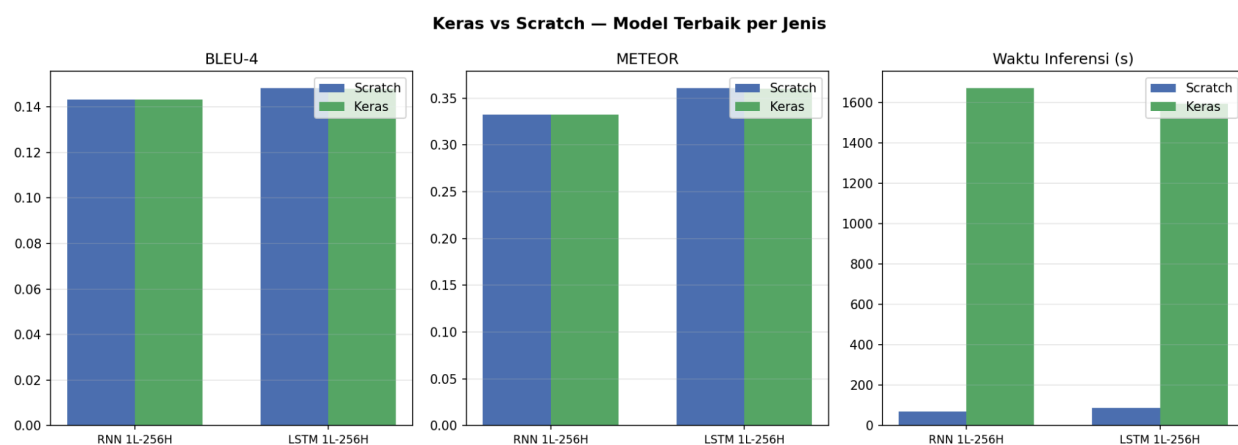
```
RNN :  
a woman in a red shirt is holding a baby in a red and red shirt  
LSTM:  
a woman in a black shirt and a woman in a black shirt is standing in front of a crowd  
GT :  
two bald drag queens in red dresses  
two bald ladies are wearing red dresses
```

Gambar 4.2.4 Analisis Kualitatif Hasil Prediksi RNN dan LSTM

4.2.6 Perbandingan Keras vs *From Scratch*

Skor performa antara Keras dan implementasi *from scratch* terbukti sangat identik. Untuk arsitektur RNN terbaik (1 layer, 256 hidden), Keras mendapat BLEU-4 0.1432 (berbanding 0.1432 *from scratch*). Untuk LSTM (1 layer, 256 hidden), Keras mendapat BLEU-4 0.1480 (berbanding 0.1481 *from scratch*). Menariknya, waktu inferensi Keras

jauh lebih lambat, memakan waktu 2126.1 detik (RNN) dan 953.3 detik (LSTM), sedangkan kode murni Python NumPy *from scratch* mampu menyelesaikan inferensi dalam waktu jauh lebih singkat yakni 83.2 detik (RNN) dan 114.6 detik (LSTM). Lambatnya Keras kemungkinan besar disebabkan oleh ketidakefisienan pemanggilan API `model.predict()` di dalam iterasi `for-loop` per kata saat siklus generatif berlangsung. Hal ini menyebabkan penumpukan pemanggilan grafik TensorFlow untuk satu langkah prediksi kecil, berbanding terbalik dengan operasi *matrix multiplication* langsung via NumPy yang *bypass overhead* framework secara keseluruhan.

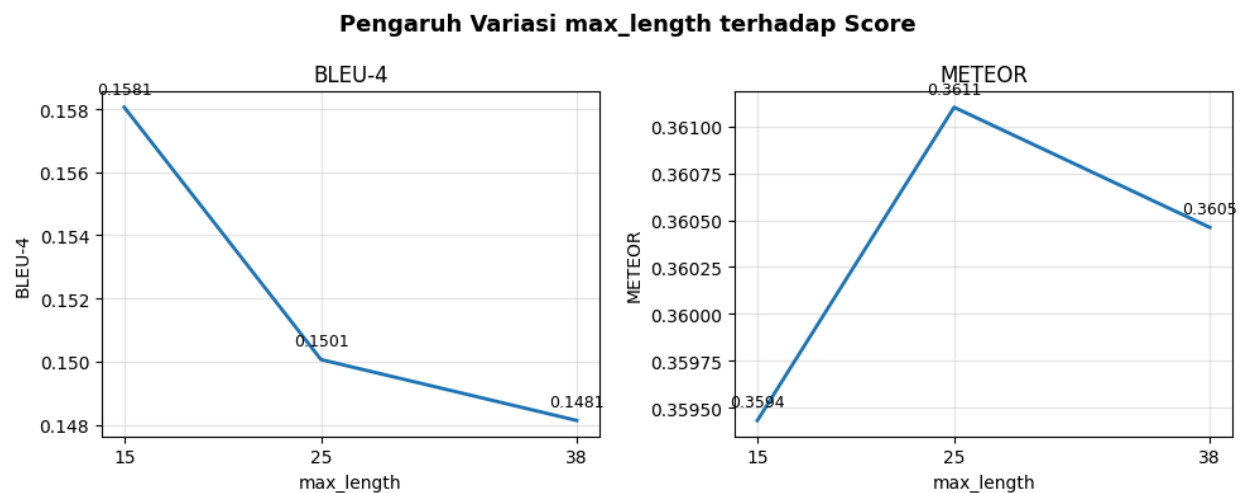


Gambar 4.2.5 Perbandingan Hasil Evaluasi Keras dan Model *From Scratch*

4.2.7 Pengaruh Panjang Maksimum Caption Terhadap BLEU-4

Dari variasi `max_length` 15, 25, dan 38, skor BLEU-4 dan METEOR tertinggi secara konsisten diperoleh pada panjang paling pendek yaitu 15 (BLEU-4: 0.1581, METEOR: 0.3594). Penurunan skor seiring bertambahnya `max_length` terjadi karena probabilitas model terperangkap dalam siklus degeneratif (mengulang kata yang sama terus menerus di akhir kalimat) menjadi lebih tinggi pada *sequence* yang panjang. Caption dengan `max_length` pendek (15) memaksa model untuk memproduksi token-token yang esensial dan padat

makna lalu segera memutusnya, yang mana hal ini sangat cocok dengan metrik evaluasi n-gram seperti BLEU-4 yang mengutamakan presisi kata. Batas max_length 15 juga relevan dengan panjang rata-rata kalimat pada dataset Flickr8k.



Gambar 4.2.6 Grafik Pengaruh Variasi max_length Terhadap Score

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil pengujian dan analisis terhadap model Convolutional Neural Network (CNN), Simple Recurrent Neural Network (RNN), dan Long Short-Term Memory (LSTM) yang telah diimplementasikan baik menggunakan Keras maupun secara *from scratch*, dapat ditarik beberapa kesimpulan utama sebagai berikut:

1. Konfigurasi dangkal dan sederhana (2 layer, filter 3×3 , MaxPooling, dan shared parameters via Conv2D) lebih optimal dan tangguh terhadap *overfitting* dibandingkan konfigurasi yang lebih kompleks (seperti 4 layer atau *non-shared parameters*).
2. LSTM secara konsisten lebih baik dari SimpleRNN (BLEU-4 0.1481 vs 0.1432). Mekanisme gerbang pada LSTM efektif mengatasi *vanishing gradient*, sehingga fitur gambar awal dari CNN tetap relevan hingga akhir proses *captioning*.
3. Arsitektur 1 layer *recurrent* dengan ukuran *hidden state* 256 adalah yang paling optimal untuk RNN dan LSTM. Penambahan *layer* justru memicu *overfitting* pada dataset ini. Selain itu, pembatasan `max_length = 15` menghasilkan skor evaluasi terbaik karena meminimalisasi repetisi kata.
4. Implementasi *forward propagation from scratch* (NumPy) menghasilkan akurasi yang identik dengan Keras untuk semua arsitektur, membuktikan validitas rumus matematis yang diterapkan.
5. Walaupun Keras unggul di eksekusi CNN karena optimisasi *hardware*, pada *task image captioning* yang iteratif, implementasi NumPy *from scratch* justru lebih cepat dieksekusi dibandingkan pemanggilan fungsi Keras yang memicu *overhead* besar.

5.2 Saran

Untuk pengembangan dan penelitian lebih lanjut terkait *image classification* maupun *image captioning*, beberapa saran yang dapat diterapkan adalah:

1. Menerapkan Dropout atau L2 Regularization untuk mengurangi *overfitting* saat meningkatkan kompleksitas model.
2. Menggunakan augmentasi spasial pada CNN untuk memperkaya variasi data latih.
3. Menggunakan arsitektur berbasis Attention untuk meningkatkan fokus spasial dari image captioning.
4. Gunakan Numba atau CuPy untuk mempercepat waktu komputasi Python murni (from scratch).
5. Memanfaatkan *word embeddings* siap pakai (seperti GloVe) untuk meningkatkan kualitas representasi kata.

BAB VI
PEMBAGIAN TUGAS

No.	NIM	Tugas
1	13523077	Implementasi <i>Recurrent Neural Network</i> (RNN), <i>Notebook</i> RNN dan LSTM
2	13523087	Implementasi <i>Convolutional Neural Network</i> (CNN), <i>Notebook</i> CNN, Bonus Visualisasi Fitur CNN
3	13523093	Implementasi <i>Long Short-Term Memory Network</i> (LSTM), Analisis Hasil Pengujian CNN, RNN, LSTM

BAB VII

REFERENSI

Chollet, F. (2021). Deep learning with Python (2nd ed.). Manning Publications.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
<https://www.deeplearningbook.org>

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>

Hodosh, M., Young, P., & Hockenmaier, J. (2013). Framing image description as a ranking task: Data, models and evaluation metrics. Journal of Artificial Intelligence Research, 47, 853–899. <https://doi.org/10.1613/jair.3994>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.
<https://doi.org/10.1109/5.726791>

BAB VIII

LAMPIRAN

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : 1. Albertus Christian Poandy/13523077
2. Grace Evelyn Simon/13523087
3. Karol Yangqian Poetrachya/13523093
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika – Komputasi (STEI-K)
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : *Convolutional Neural Network dan Recurrent Neural Network*

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/Tidak*	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Mei 2026



Albertus Christian Poandy
13523077



Grace Evelyn Simon
13523087



Karol Yangqian Poetrachya
13523093